MapReduce: Simplified Data Processing on Large Clusters

Authors: Jeffrey Dean and Sanjay Ghemawat (Google)

Presenter: Hasan Arı

1 Introduction

Context:

- Google processes massive amounts of raw data (crawled documents, web logs).
- Computations are conceptually straightforward (e.g., inverted indices).

The Problem:

- Input volume requires distribution across thousands of machines.
- Complexity: Issues of parallelization, data distribution, and fault tolerance obscure the simple computation logic.

The Solution:

- MapReduce: A new abstraction that hides the messy details of parallelization, fault tolerance, and load balancing in a library.

2 Programming Model

Concept

Inspired by map and reduce primitives in functional languages

Map Function

- Takes an input pair and produces a set of intermediate key/value pairs
- Map (k1, v1) → list(k2, v2)

The Library's Role

Groups together all intermediate values associated with the same intermediate key I.

Reduce Function

Accepts an intermediate key *I* and a set of values for that key

- Merges values to form a possibly smaller set of values

2.1 Example

Problem: Counting the number of occurrences of each word in a large collection of documents.

Execution: User links code with the MapReduce library and passes a specification object.

```
map(String key, String value):
    // key: document name
    // value: document contents
    for each word w in value:
        EmitIntermediate(w, "1");

reduce(String key, Iterator values):
    // key: a word
    // values: a list of counts
    int result = 0;
    for each v in values:
        result += ParseInt(v);
    Emit(AsString(result));
```

2.2 Types

Type Signatures: Conceptual types for user-supplied map and reduce functions.

map	(k1,v1)	→ list(k2,v2)
reduce	(k2,list(v2))	→ list(v2)

Domain Distinction:

Input: Drawn from a different domain than the output (*k1, v1 / k2, v2*).

- Intermediate: Same domain as the output keys and values.

3 Implementation

Target Environment

Large clusters of commodity PCs connected with switched Gigabit Ethernet.

Hardware Specs

- Dual-processor x86, Linux, 2-4GB memory per machine.
- Storage: Inexpensive IDE disks attached directly to machines.

System Stack

- GFS (Google File System): Manages data stored on these disks using replication for reliability.
- Scheduling: Users submit jobs to a scheduling system which maps tasks to available machines.

3.1.a Execution Overview

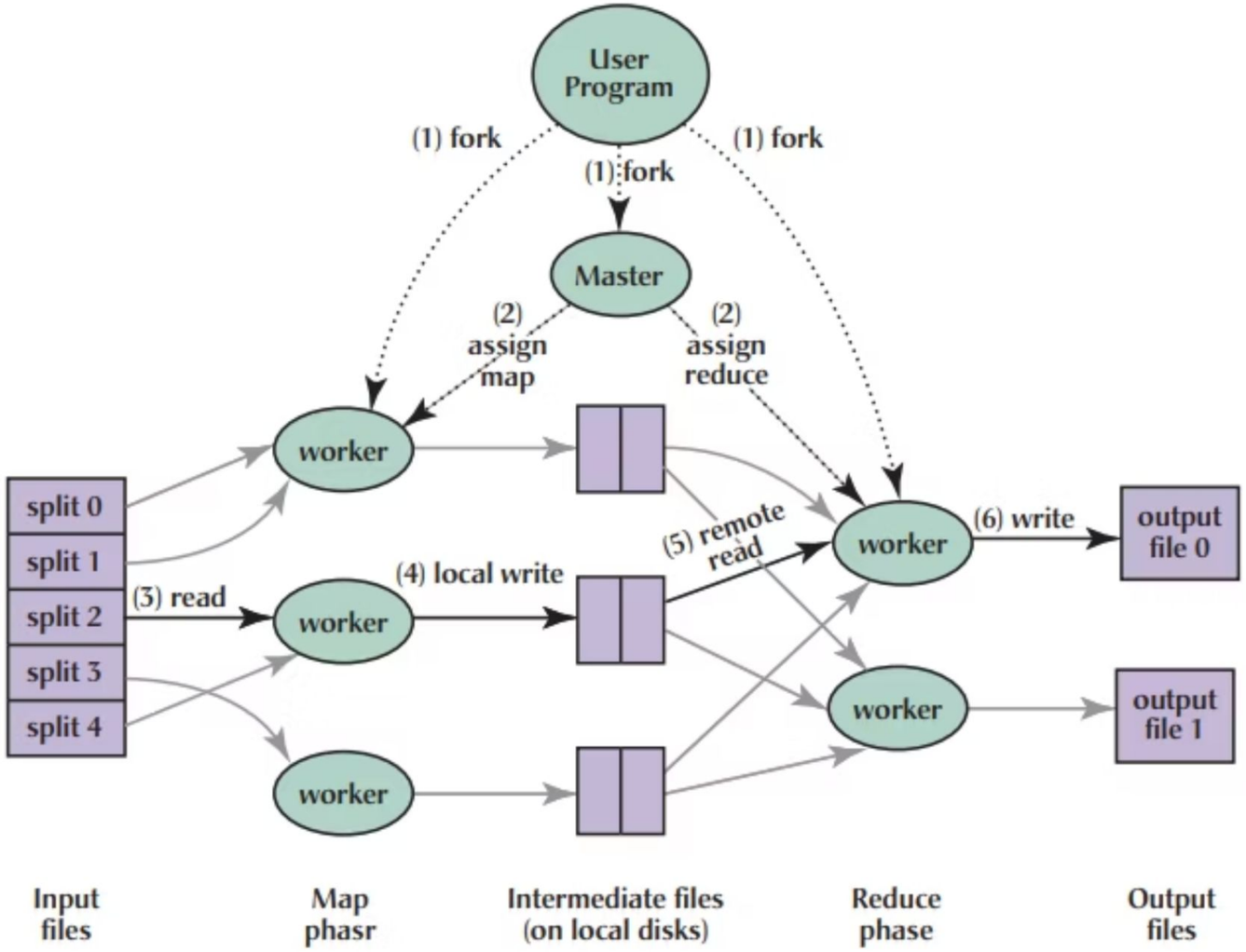
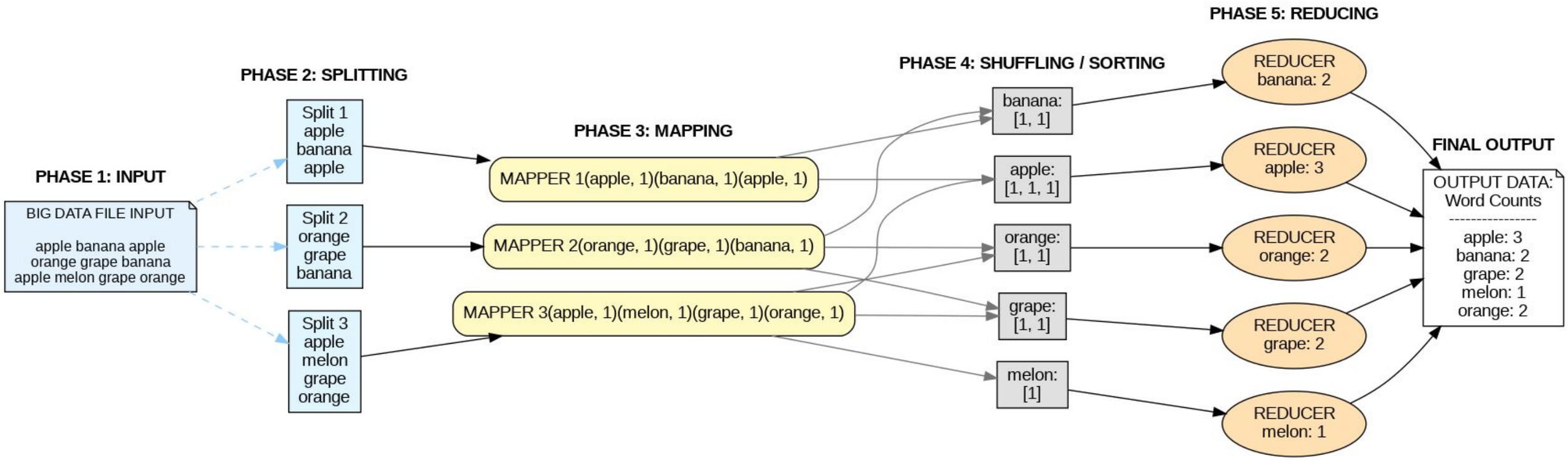


Fig. 1. Execution overview.

3.1.b Word Count example



3.1.c – The Shuffle Process (Who does what?)

Map Worker's Role (Preparation):

- Buffers output in memory.

Writes to local disk, partitioned into *R* regions.

Reduce Worker's Role (Execution):

- Remote Read: Reads buffered data from Map workers' local disks via RPC.
- Sort: Groups occurrences of the same key together by sorting.

Key Takeaway: The Reduce worker is the active agent in the shuffle phase (Pull model).

3.2 Master Data Structures

01

Task State

Stores the state (idle, in-progress, or completed) and worker identity for each map and reduce task.

02

Location of Data

The master propagates the location of intermediate file regions from map tasks to reduce tasks.

03

File Metadata

- For each completed map task, the master stores the locations and sizes of the R intermediate file regions.
- Information is pushed incrementally to workers with in-progress reduce tasks.

3.3 Fault Tolerance

Handling Worker Failures:

- Master pings workers periodically; no response marks worker as failed.
- Map Tasks: Completed map tasks are re-executed on failure because their output is stored on the failed machine's inaccessible local disk.
- Reduce Tasks: Completed reduce tasks do not need re-execution (output is stored in global file system).

Resilience:

- Capable of handling large-scale failures (e.g., losing 80 machines simultaneously) without stopping the job.

Semantics:

- Relies on atomic commits of task outputs to guarantee deterministic results equivalent to sequential execution.

3.4 Locality

Constraint: Network bandwidth is a relatively scarce resource.

Optimization Strategy

Leverage GFS (Google File System) which stores input data on local disks of cluster machines.

Scheduling

Master attempts to schedule a map task on a machine that contains a replica of the corresponding input data. Failing that, it schedules near a replica (e.g., same network switch).

Benefit

Most input data is read locally and consumes no network bandwidth.

3.5 Task Granularity

Concept: Subdividing the map phase into *M* pieces and reduce phase into *R* pieces.

Guideline: *M* and *R* should be much larger than the number of worker machines.

Benefits:

- Dynamic Load Balancing: Faster workers process more tasks.
- Faster Recovery: Tasks from a failed worker can be spread out across many other workers.

Practical Choices:

M chosen so each task is ~16MB to 64MB (optimizes locality).

R is a small multiple of the number of worker machines.

3.6 Backup Tasks

The Problem: Stragglers

A machine that takes an unusually long time to complete one of the last few tasks (e.g., due to bad disk or CPU competition).

The Solution: Backup Executions

When operation is close to completion, the master schedules backup executions of remaining in-progress tasks.

Outcome:

- Task is marked completed when either the primary or backup execution finishes.
- Significantly reduces completion time (e.g., Sort program takes 44% longer without backups).

4 Refinements (Partitioning & Ordering)

Custom Partitioning

- Default is hash(key) mod R.
- Users can specify a custom partitioning function (e.g., hash by hostname only) to control which keys go to which Reduce task.

Ordering Guarantees

- Within each Reduce partition, intermediate key/value pairs are guaranteed to be processed in increasing key order.
- Essential for generating sorted output files efficiently.

4 Refinements (Combiner Function)

The Problem:

Map tasks often generate massive repetition (e.g., <the, 1> thousands of times). Sending all across the network is inefficient.

The Solution: Combiner Function:

User-specified function that does partial merging of data before it leaves the Map machine.

Benefit:

- Drastically reduces the amount of intermediate data transferred across the network.
- Example: Sends one <the, 1000> instead of 1,000 records of <the, 1>

5 Performance (Cluster Configuration)

Cluster Setup

~1800 machines.

Machine Specs

- 2GHz Intel Xeon processors, 4GB Memory.
- Two 160GB IDE disks per machine.
- Gigabit Ethernet link.

Network

- Two-level tree-switched network.
- Aggregate bandwidth at root: ~100-200 Gbps.

Timing

executed during off-peak hours (weekend afternoon).

5.2 Grep Performance

Task: Scan *10^{10}* 100-byte records (~1 TB) for a rare pattern.

Graph Analysis (Figure 2):

- Rate: Input processing peaks at over 30 GB/s12.
- Startup Overhead: Includes propagation of program and GFS interaction (~60 seconds).
- Completion: Entire computation takes ~150 seconds.

Takeaway: Demonstrates massive parallelism in data reading.

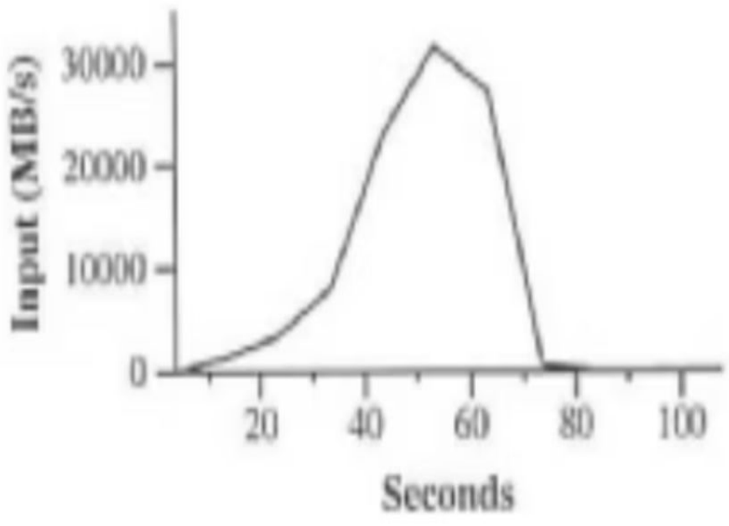


Fig. 2. Data transfer rate over time (mr-grep).

5.3 Sort Performance

Task: Sort *10^{10}* 100-byte records (~1 TB)18.

Graph Analysis (Figure 3):

- **Input Rate**: Peaks at ~13 GB/s. Lower than Grep because Map tasks write output to local disk.
- **Shuffle Phase**: Data transfer from Map to Reduce workers.
- **Output Phase**: Writes 2 replicas to GFS for reliability.

Effect of Backup Tasks: Disabling backup tasks causes the process to take 44% longer due to stragglers.

Figure 2: Data transfer rate over time (mr-grep)

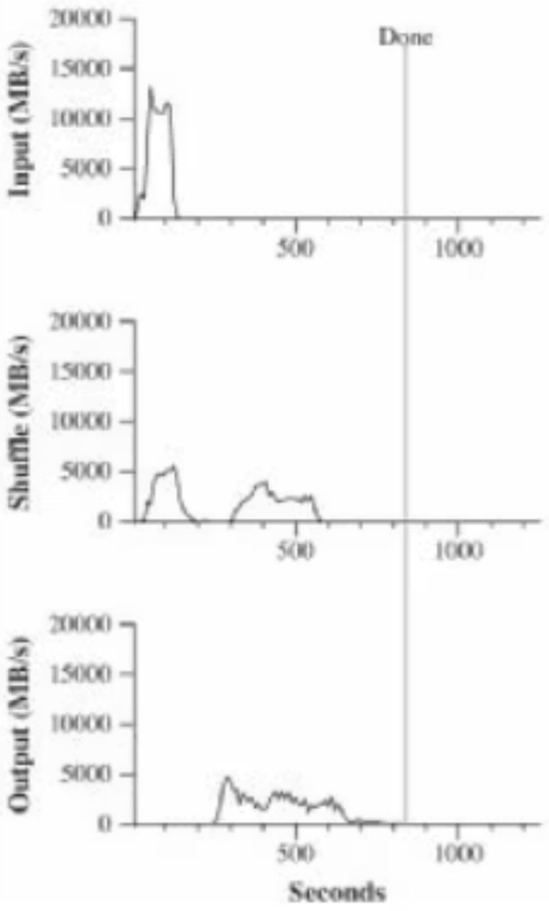


Fig. 3. Data transfer rate over time (mr-sort).

6 Experience (Usage at Google)

Adoption: Grew from 0 to thousands of distinct MapReduce programs within a few years.

Use Cases:

- Large-scale machine learning problems.
- Clustering for Google News.
- Extracting popular queries (Google Trends).

Case Study: Indexing System Rewrite:

- Rewrote the production indexing system using MapReduce.
- Result: Code size dropped from ~3800 to ~700 lines of C++
- Improved fault tolerance and performance.

	Aug. '04	Mar. '06	Sep. '07
Number of jobs (1000s)	29	171	2,217
Avg. completion time (secs)	634	874	395
Machine years used	217	2,002	11,081
map input data (TB)	3,288	52,254	403,152
map output data (TB)	758	6,743	34,774
reduce output data (TB)	193	2,970	14,018
Avg. machines per job	157	268	394
Unique implementations			
map	395	1958	4083
reduce	269	1208	2418

7 - Conclusion

Key Success Factors:

1. Easy to Use: Hides complexity (parallelization, fault tolerance, locality) from the programmer.
2. Expressive: Applicable to a wide variety of real-world problems (sorting, data mining, ML).
3. Scalable: Scales efficiently to clusters comprising thousands of machines.

Final Thought: By restricting the programming model, MapReduce makes it easy to parallelize computations and make them fault-tolerant.